第二讲教学设计：第一个Servlet—开启动态内容的大门 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: 第一个Servlet：开启动态内容的大门
- **授课时长**: 20分钟
- **适用对象**: 已了解HTTP和Tomcat基础的Java Web开发初学者

一、教学目标

知识目标

1. 理解Servlet在Web应用中作为"后端大脑"的核心角色。
2. 掌握Servlet的生命周期（Lifecycle: `init`，`service`，`destroy`）及其意义。
3. 掌握 `@WebServlet` 注解（Annotation）的核心用法，用于将URL请求映射（Mapping）到Servlet类。

能力目标

1. 能够编写一个继承自 `HttpServlet` 的简单Servlet类。
2. 能够重写 `doGet` 方法，使用 `request.getParameter()` 从URL获取参数，并使用 `response.getWriter()` 向浏览器输出动态内容。
3. 能够将带有Servlet的Web应用打包成WAR文件（或目录）并部署到Tomcat。

素质目标

1. 培养从静态思维到动态、交互式编程思维的转变。
2. 建立对客户端-服务器端数据交互流程的清晰认识。
3. **[思政融合]** 结合Servlet处理用户输入，强调网络安全和信息处理的责任与规范，为讲解自主创新和国产软件安全打下基础。

二、教学重点与难点

教学重点

- Servlet的生命周期及其高效性（一次实例化，多次服务）。
- `HttpServlet` 的核心方法： `doGet()` 和 `doPost()` 的区别与使用。
- 通过 `@WebServlet` 注解实现URL到Servlet的映射。
- `HttpServletRequest` （请求）和 `HttpServletResponse` （响应）两个核心对象的基本使用。

教学难点

- **概念难点**: 理解Servlet实例由Tomcat（容器）创建和管理，而非由我们手动 `new` 出来，以及为何它是单实例、多线程的。
- **实践难点**: 将HTML表单的 `action` 属性与Servlet的 `@WebServlet` 路径正确对应起来，完成第一次前后端交互。

三、教学内容与时间安排

第一部分：问题引入 - 会"说话"的网页 (3分钟)

内容要点

1. **回顾与设问 (Review & Question):**
   - "上节课，我们成功部署了一个静态HTML页面。但它就像一张海报，只会展示，不会互动。如果我们想让网页响应我们的操作，比如一个登录，该怎么办？"
2. **趣味案例引入 (Engaging Example):**
   - 展示一个极其简单的HTML页面 `login.html`，设计成一个"特工登录"界面。

```
<!-- login.html -->
<h2>特工登录 (Secret Agent Login)</h2>
<form action="login" method="get">
    输入你的代号 (Enter Your Codename):
    <input type="text" name="agentName" />
    <input type="submit" value="确认身份 (Confirm Identity)" />
</form>
```

   - 现场演示：在浏览器打开此页面，输入"007"，点击按钮。结果是什么？—— 一个 `404 Not Found` 错误！
   - **提出核心问题**: "我们的服务器(Tomcat)为什么不认识 `/login` 这个地址？我们如何创造一个'接头人'，让它能接收到'007'这个代号，并给出响应？"
3. **引出主角 (Introducing the Protagonist):**
   - 这个服务器端的"接头人"或"大脑"，在Java Web世界里，就是 **Servlet**。它的使命，就是接收请求、处理数据、并做出响应。

教学方法

- 场景驱动法，用一个有趣的小案例制造"痛点"。
- 悬念式提问，引导学生思考解决方案。

第二部分：编写你的第一个Servlet (8分钟)

内容要点

1. **创建Servlet类 (Creating the Servlet):**
   - 在IDE中创建一个新的Java类 `LoginServlet`。
   - **关键一步**: `public class LoginServlet extends HttpServlet`，强调继承 `HttpServlet` 是成为一个处理HTTP请求的Servlet的基础。
2. **实现业务逻辑 (Implementing Business Logic):**
   - 重写 `doGet` 方法。解释：因为我们的HTML表单 `method="get"`，所以Tomcat会调用 `doGet` 方法。

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse
response)
```

```
        throws ServletException, IOException {
    // 业务逻辑写在这里
}
```

3. **接收数据与做出响应 (Receiving Data & Responding):**
   - **接收**: `String name = request.getParameter("agentName");` 讲解 `request` 对象是封装了所有请求信息的"包裹"，`getParameter` 就是从包裹里按名字取出物品。
   - **响应**:

     ```
     // 为了让浏览器能正确解析中文，设置响应编码
     response.setContentType("text/html;charset=UTF-8");
     PrintWriter out = response.getWriter(); // 拿到一个"画笔"
     out.println("<h1>身份确认!</h1>");
     out.println("<h2>欢迎你, " + name + " 特工!</h2>");
     out.println("<h3>你的下一个任务是：精通Servlet!</h3>");
     ```

     讲解 `response` 对象是我们要发回的"回信"，`getWriter` 就是拿到写信的笔。
4. **建立连接 - 注解 (Making the Connection - Annotation):**
   - 在 `LoginServlet` 类定义的上方添加注解：`@WebServlet("/login")` 。
   - **强调**: 这个注解就像是给我们的"接头人"分配了一个独一无二的接头暗号（`/login`）。当Tomcat看到有请求访问这个地址时，就会准确地找到并调用 `LoginServlet` 。
5. **现场联调 (Live Debugging & Demo):**
   - 重新部署应用。
   - 再次访问 `login.html` ，输入代号，点击提交。
   - 屏幕上成功显示出动态生成的欢迎语！第一次前后端交互成功！

教学方法

- **现场编码 (Live Coding)**，分步讲解，让学生看到从无到有的过程。
- **代码-功能对应讲解**: 每一行代码都解释其在整个交互流程中的作用。

第三部分：Servlet的生命周期 - 高效的秘密 (6分钟)

内容要点

1. **引入思考**: "我们刷新了好几次欢迎页面，`LoginServlet` 是不是每次都被重新创建了一个对象？如果每秒有一千个请求，服务器岂不是要创建一千个对象，那服务器不就崩了？"
2. **生命周期讲解 (Lifecycle Explanation):**
   - **比喻**: Servlet就像一个餐厅里**唯一**的、技艺高超的厨师。
   - **1. 实例化与初始化 (Instantiation & Initialization - `init()`):**
     - 餐厅（Tomcat）第一次接到"炒饭"订单（第一个请求）时，才把这位厨师请来上班并让他熟悉厨房（`init()` 方法被调用）。**这个过程只发生一次!**
     - *实践验证*: 在 `LoginServlet` 中添加 `init()` 方法和 `System.out.println("厨师上班了！");`
   - **2. 服务 (Service - `service()` / `doGet()` / `doPost()`):**
     - 之后，每一份新的"炒饭"订单（每一个请求），都是由**这位厨师**亲自完成。他会不停地服务，但人还是同一个人。

        - *实践验证*: 在 `doGet()` 方法中添加 `System.out.println("厨师炒了一份饭！");` 。刷新浏览器，控制台会不断打印"炒了一份饭"，但"厨师上班了"只打印一次。
    - **3. 销毁 (Destroy - `destroy()`):**
        - 当餐厅关门（服务器关闭）时，这位厨师才下班回家（`destroy()` 方法被调用）。**这个过程也只发生一次！**
        - *实践验证*: 添加 `destroy()` 方法和 `System.out.println("厨师下班了！");` 。关闭Tomcat服务器时，控制台会打印此信息。

3. **核心结论**: Servlet是**单实例、多线程**的。这种设计极其高效，因为它避免了频繁创建和销毁对象的开销。

教学方法

- 生动的比喻，将抽象概念具体化。
- 通过在控制台打印日志，让学生"亲眼看到"生命周期方法的调用时机。

第四部分：总结与新的挑战 (3分钟)

总结要点

1. **Servlet是"动态大脑"**: 它是处理客户端请求的Java类。
2. **注解是"路标"**: `@WebServlet` 将URL和Servlet关联起来。
3. **生命周期是"效率之源"**: `init()` 一次，`service()` 多次，`destroy()` 一次。
4. `Request` **和** `Response` **是"左膀右臂"**: 一个用来接收，一个用来发送。

提出新的挑战 (Paving the Way for JSP)

- 展示 `doGet` 方法里用 `out.println()` 拼接HTML的代码。
- **提问**: "同学们，看看我们的代码，如果欢迎页面非常复杂，有上百行HTML，那我们的Servlet类会变成什么样？是不是像一碗混在一起的意大利面，难以阅读和维护？如果一个UI设计师想修改页面样式，他是不是必须得懂Java，还得修改我们的Servlet类？这太可怕了！"
- **展望**: "我们需要一种能将"前端的归前端，后端的归后端"的技术。下一节课，我们将学习JSP，它将彻底解决这个问题，让我们的视图层变得优雅和专业！"

四、教学方法与手段

教学方法

1. **案例驱动法 (Case-Driven Method)**: 以"特工登录"趣味案例引导学习
2. **现场编码 (Live Coding)**: 分步编写Servlet代码
3. **比喻教学法 (Analogy)**: 用"厨师"比喻Servlet生命周期
4. **对比分析法 (Comparison)**: 对比静态页面与动态Servlet
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（如Lifecycle, Annotation, Mapping）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述Servlet工作流程
    - 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)

- 浏览器开发者工具 (Browser DevTools)
- PPT/Marp课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 能否跟上编码节奏，理解 `request.getParameter` 的用法。
- **终结性评价**: 能否独立编写一个简单的Servlet响应请求；能否用厨师的比喻复述Servlet的生命周期。

六、课后拓展

思考题

1. `doGet` 和 `doPost` 方法在使用上有什么本质区别？（提示：思考数据是如何提交给服务器的）
2. 如果 `@WebServlet("/login")` 和HTML表单的 `action="login"` 都改成 `"/user/login"`，程序还能正常工作吗？为什么？

实践作业

1. **扩展登录功能**: 给 `login.html` 增加一个密码输入框 (`<input type="password" name="agentPwd">`)。
2. **修改Servlet**: 在 `LoginServlet` 中也获取密码。
3. **增加判断**: 判断用户名是否为"007"，密码是否为"123456"。如果是，则显示欢迎；如果不是，则显示"身份认证失败，你不是我们的特工！"。